Chapter 2 Python for Natural Language Processing

Regular Expressions (RegExes)

Create string patterns and use them for searching and substituting specific pattern matches in textual data.

Tools

Python offers a rich module named `re` for creating and using regular expressions.

```
import re
```

https://docs.python.org/3/library/re.html

https://regex101.com/

Some Important Flags

Can be used in `re` function calls.

```
re.I
re.IGNORECASE
# is used to match patterns ignoring case sensitivity.
```

```
re.S
re.DOTALL
# causes the period (.) character (or dot) to match any character,
# including new lines.
```

Popular Rules

`.` for matching a single character

`^` for matching the start of the string

`$` for matching the end of the string

`*` for matching **zero or more** cases of the previously mentioned regex before the `*` symbol in the pattern

`+` for matching **one or more** cases of the previously mentioned regex before the `+` symbol in the pattern

`?` for matching **zero or one** case of the previously mentioned regex before the `?` symbol in the pattern

`.` for matching a single character

Regex: `h.t`

Test Case	Matches
`hat`	hat
`hit`	hit
`hot`	hot
`h.t`	h.t

`^` for matching the start of the string

Regex: `^The`

Test Case	Matches
`The quick brown fox`	The
`A quick fox`	No match
`The lazy dog`	The

`$` for matching the end of the string

Regex: `end$`

Test Case	Matches
`This is the end`	end
`This is not the end`	end
`The end`	end

`*` for matching **zero or more** cases of the previously mentioned regex before the `*` symbol in the pattern

Regex: `ab*c`

Test Case	Matches
`ac`	ac
`abc`	abc
`abbbc`	abbbc
`a b c`	No match

`+` for matching **one or more** cases of the previously mentioned regex before the `+` symbol in the pattern

Regex: `ab+`

Test Case	Matches
`a`	No match
`ab`	ab
`abb`	abb
`abbb`	abbb

`?` for matching **zero or one** case of the previously mentioned regex before the `?` symbol in the pattern

Regex: `colou?r`

Test Case	Matches
`color`	color
`colour`	colour
`colur`	No match

Popular Rules (cont'd)

`[...]` for matching any one of the set of characters inside the square brackets

`[^...]` for matching a character not present in the square brackets after the `^` symbol

`|` denotes the OR operator for matching either the preceding or the next regex

`\d` for matching decimal digits, which are also depicted as `[0-9]`

`\D` for matching non-digits, also depicted as `[^0-9]`

`[...]` for matching any one of the set of characters inside the square brackets

Regex: `[aeiou]`

Test Case	Matches
`hello`	e, o
`regex`	e, e
`world`	o

`[^...]` for matching a character not present in the square brackets after the ˆ symbol

Regex: `[^aeiou]`

Test Case	Matches
`hello`	h, l, l
`regex`	r, g, x
`world`	w, r, l, d

`|` denotes the OR operator for matching either the preceding or the next regex

Regex: `cat|dog`

Test Case	Matches
`I have a cat.`	cat
`I have a dog.`	dog
`I have a bat.`	No match

`\d` for matching decimal digits, which are also depicted as `[0-9]`

Regex: `\d+`

Test Case	Matches
`123`	123
`abc`	No match
`456def`	456

`\D` for matching non-digits, also depicted as `[^0-9]`

Regex: `\D+`

Test Case	Matches
`123`	No match
`abc`	abc
`456def`	def

Popular Rules (cont'd)

`\s` for matching whitespace characters

`\S` for matching non-whitespace characters

`\w` for matching word characters: letters, digits and underscore

`\W` for matching non-word characters, i.e. anything `\w` does not match

In Python 3 these are Unicode-aware: `\w` also matches letters such as `è`. They are only `[a-zA-Z0-9_]` / `[^a-zA-Z0-9_]` under the `re.ASCII` flag, or for bytes patterns.

`\s` for matching whitespace characters

Regex: `\s`

Test Case	Matches
`hello world`	(space)
`one  two`	(space), (space), (space), (space)
`no_space`	No match

`\S` for matching non-whitespace characters

Regex: `\S+`

Test Case	Matches
`hello world`	hello, world
`one  two`	one, two
`no_space`	no_space

`\w` for matching word characters (letters, digits, underscore)

Regex: `\w+`

Test Case	Matches
`hello_world`	hello_world
`regex123`	regex123
`!@#`	No match

`\W` for matching non-word characters

Regex: `\W+`

Test Case	Matches
`hello_world`	No match
`regex123!`	!
`!@#`	!@#

Main RE Methods

```
re.compile()
```

compiles a specified regular expression pattern into a regular expression object, which can be used for matching and searching. Takes a pattern and optional flags as input (see above).

```
re.match()
```

used to match patterns at the **beginning** of strings.

```
re.search()
```

used to match patterns occurring at **any position** in the string.

Main RE Methods (cont'd)

```
re.findall()
```

returns all non-overlapping matches of the specified regex pattern in the string.

```
re.finditer()
```

returns all matched instances in the form of an iterator, for a specific pattern in a string when scanned from left to right.

```
re.sub()
```

used to substitute a specified regex pattern in a string with a replacement string. If there are **overlapping** occurrences, it only substitutes *each* left-most occurrence of the pattern in the string.

Examples

```
# creating some strings
s1 = 'Python is an excellent language'
```

```
s2 = 'I love the Python language. \
I also use Python to build applications at work!'
```

```
pattern = 'python'
```

Matching

`match` only returns a match if regex match is found at the beginning of the string.

```
print(re.match(pattern, s1))
# pattern is in lower case hence ignore
```

```
# case flag helps in matching same pattern with different cases
print(re.match(pattern, s1, flags=re.IGNORECASE))
```

```
# printing matched string and its indices in the original string
m = re.match(pattern, s1, flags=re.IGNORECASE)
```

```
print('Found match "{}" ranging from index {} - {} in the string "{}"' \
      .format(m.group(0), m.start(), m.end(), s1))
```

```
# match does not work when pattern is not there in the beginning of string s2
print(re.match(pattern, s2, re.IGNORECASE))
```

Find/Search

Let's now look at some examples that illustrate how the find(...) and search(...) methods work in regular expressions:

Only find first:

```
re.search(pattern, s2, re.IGNORECASE)
```

Find all, but only shows matches themselves:

```
re.findall(pattern, s2, re.IGNORECASE)
```

Find/Search (cont'd)

Allows as to iterate over all matches with their locations:

```
match_objs = re.finditer(pattern, s2, re.IGNORECASE)
match_objs
```

```
print("String:", s2)
```

```
for m in match_objs:
    print('Found match "{}" ranging from index {} - {}' \
          .format(m.group(0), m.start(), m.end()))
```

Substitution

Illustrating pattern substitution using sub and subn methods

```
# substitute all:
re.sub(pattern, 'Java', s2, flags=re.IGNORECASE)
```

```
# substitute all, and report the number of substitutions (n):
# tuple: (new_string, n)
re.subn(pattern, 'Java', s2, flags=re.IGNORECASE)
```

Example:

Replace all Email Addresses with a Placeholder Token

```
text = "Contact us at support@rotman.utoronto.ca or sales@example.com for info."
```

```
pattern = r'[\w\.-]+@[\w\.-]+\.\w+'
```

```
re.subn(pattern, 'EMAIL', text)
```

Dealing with Unicode Matching using Regexes

```
s = u'H\u00e8llo! this is Python '
s
```

```
re.findall(r'\w+', s)
```

```
re.findall(r"[A-Z]\w+", s)
```

```
emoji_pattern = r"[' \U0001F300-\U0001F5FF'|'\U0001F600-\U0001F64F'|\
'\U0001F680-\U0001F6FF'|'\u2600-\u26FF\u2700-\u27BF']"
```

```
re.findall(emoji_pattern, s)
```

Additional Readings

Text Analytics with Python: A Practitioner's Guide to Natural Language Processing Paperback, 2019, by Dipanjan Sarkar

https://docs.python.org/3/library/re.html

https://regex101.com/